

— S8

풀스택 배포와 분석 셋업

검증된 MVP를 공개 URL로 노출 + 사용자 행동 자동 수집

— 이론 학습 목표

- 01 FE/BE 분리 배포의 신뢰 경계 의미 설명
- 02 HTTP·도메인·포트·환경 변수·빌드/배포·API의 최소 개념 인식
- 03 환경 변수 3종류 구분 (BE 비밀 / FE 공개 / 로컬용)
- 04 CORS가 왜 존재하고 어떻게 다룰 것인가의 기획 결정
- 05 분석 도구 3종이 각각 답하는 다른 질문 인식
- 06 사용자 들어오기 전 분석 셋업의 시점성 이해

— 이론 1

풀스택 구조와 FE/BE 분리

사용자 화면과 서버가 분리된 구조 — 비밀은 어디에?

- 01 FE와 BE의 차이를 설명한다
- 02 신뢰 경계의 의미를 안다
- 03 비밀이 BE에 있어야 하는 이유를 풀이한다

풀스택의 정체

사용자가 보는 화면과 실제 일을 처리하는 서버가 분리된 구조

영역	위치	역할
FE (Frontend)	사용자 브라우저	화면 표시, 입력 받기
BE (Backend)	외부 서버	비밀 보관, 로직 처리, LLM 호출

두 영역이 HTTP로 통신.

본 학습에서 BE가 필요한 이유

시드 6개 모두 LLM API 호출 핵심 기능 보유 — 핵심 위험

- 1 FE에서 직접 LLM API 호출
- 2 API 키가 브라우저 코드에 노출
- 3 누구나 개발자 도구로 코드 조회 가능
- 4 봇이 키 발견 → 무한 호출
- 5 다음 달 청구서 폭발

시드 6번 시나리오: 셀러 1명이 카피 1회 생성하면 토큰 수천 개. 봇이 1초당 100번 호출하면 시간당 36만 회 + API 비용 폭발.

단정 1

**비밀은 BE에 둔다.
FE는 BE에게 물어본다.**

신뢰 경계 (Trust Boundary)

영역	신뢰 가능성	보관 가능한 것
FE	차단	화면 코드, 공개 URL
BE	허용	API 키, DB 인증, 외부 서비스 인증

브라우저로 전달되는 모든 코드는 사용자가 볼 수 있다. FE에 둔 것은 공개된 것과 같다.

이론 1 · 비유

식당 비유

FE = 홀, BE = 주방

FE = 홀 (HALL)

손님(사용자)

- 홀에서 주문 (FE에서 입력)
- 홀에서 음식 받음 (FE에서 결과 표시)
- 주방 못 들어감 (BE 직접 접근 불가)

BE = 주방 (KITCHEN)

레시피와 재료 (API 키, 비밀번호)

- 주방에만 있음 (BE에만 보관)
- 손님에게 안 보임 (브라우저 조회 불가)

분리 배포의 정의

FE와 BE를 다른 서버에 배포하는 방식

항목	본 학습 선택
FE 플랫폼	Vercel
BE 플랫폼	Railway
통신 방식	HTTPS 요청
도메인	서로 다름

Vercel 은 FE 배포 플랫폼 (Next.js 통합 우수). Railway 는 BE 배포 플랫폼 (서버 + DB + 워커).

안티패턴 vs 분리 배포

안티패턴

"OpenAI API 키를 FE 코드에 넣고
.env.local로 관리하면 안전하지 않은가"
→ .env.local의 NEXT_PUBLIC_* 변수는 브라우저로 노출됨
→ 위험

분리 배포

"API 키는 BE에만 둔다.
FE는 BE의 엔드포인트에 요청하고,
BE가 LLM을 호출한다"

FE 직접 호출 vs BE 경유

비교	FE에서 LLM 직접 호출	BE 경유
API 키 노출	브라우저 코드에 노출	BE에만 보관
봇 호출 위험	매우 높음	차단 가능
rate limiting	어려움	가능
비용 통제	불가능	가능

01 챕터 용어 해설

용어	정의
FE (Frontend)	사용자가 직접 보는 화면. 브라우저 실행
BE (Backend)	사용자에게 안 보이는 서버. 비밀과 로직 보관
분리 배포	FE와 BE를 다른 서버에 배포
신뢰 경계	FE는 신뢰 X, BE는 신뢰 O

— 이론 2

이번 단위 최소 개발 지식

기획자가 알아야 할 최소 개발 어휘 — 코딩이 아닌 어휘

- 01 본 자료의 실습이 어떤 개발 개념 위에서 흐르는지 인식
- 02 HTTP·도메인·포트·환경 변수·빌드/배포·API 최소 개념
- 03 막혔을 때 어디를 점검할지 추측 가능

왜 본 챕터가 필요한가

본 자료는 개발 지식이 필요한 단위. 막힘 줄이는 핵심은 어휘

모르면

에러 메시지 못 읽음 → 추측 명령 → 결과 어긋남

알면

어디가 문제인지 짐작 → 정확한 명령 → 1회 해결

본 챕터의 목표는 코딩 능력이 아닌 개발 어휘. 단어와 개념이 머리에 있으면 AI와의 대화가 정확해진다.

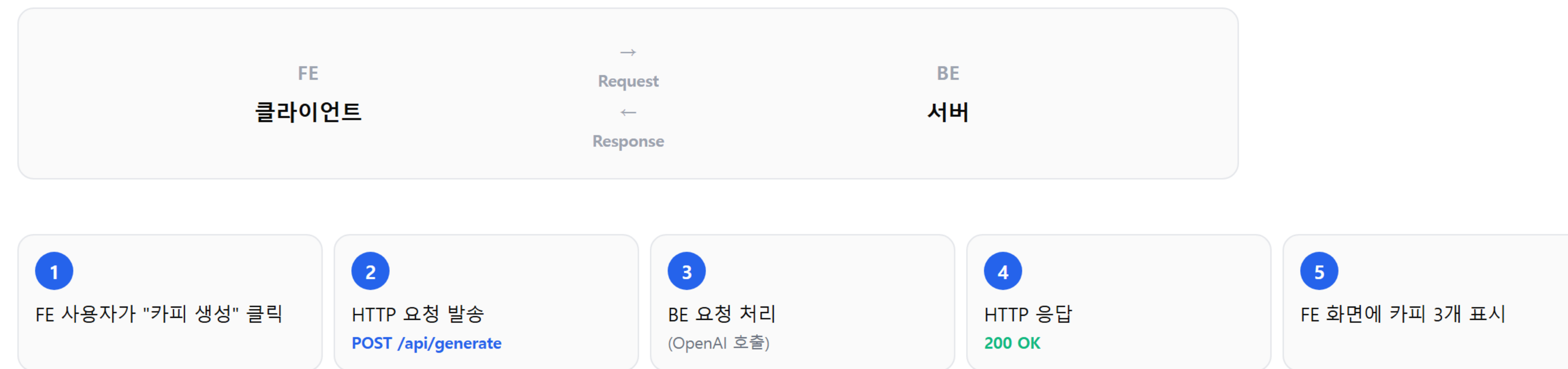
최소 지식 6가지

본 학습이 깔고 가는 개발 어휘

#	개념	이유
1	HTTP 요청·응답	FE와 BE의 통신 방식
2	도메인 (Domain)	FE와 BE의 위치
3	포트 (Port)	서버 안의 문
4	환경 변수	코드 밖의 주입 값
5	빌드와 배포	코드를 공개하는 두 단계
6	API 엔드포인트	BE가 받는 주소

1. HTTP 요청·응답

FE와 BE의 통신은 HTTP 요청·응답으로 일어남



HTTP 메서드 4가지(GET/POST/PUT/DELETE) 중 본 학습은 주로 POST 사용. 데이터를 보내고 응답받는 패턴.

2. 도메인 (Domain)

서비스의 주소. 사람이 읽을 수 있는 형태

FE 위치

`https://myapp.vercel.app`

도메인 (FE 위치)

BE 위치

`https://myapp-production.up.railway.app`

도메인 (BE 위치)

도메인이 다르면 서로 다른 서비스. 본 학습은 FE와 BE의 도메인이 다르게 발급됨 → 이것이 CORS 문제의 출발점.

3. 포트 (Port)

서버 내부의 통신 문. 한 서버에 여러 서비스 동시 운영 가능

로컬 개발 환경:

- FE: http://localhost:3000 (3000번 문)

- BE: http://localhost:8000 (8000번 문)

↑

localhost는 같은데 포트가 다름

배포 환경에서는 플랫폼이 포트 자동 관리. 본 학습에서 알아야 할 것:

환경	포트 처리
로컬	직접 명시 (3000, 8000 등)
Railway	<code>PORT</code> 환경 변수 자동 주입
Vercel	자동 (신경 안 써도 됨)

Railway 빌드 실패의 흔한 원인: BE 코드가 고정 포트(예: 8000)를 쓰면 Railway가 못 알아봄. `process.env.PORT` 로 받아야 함.

4. 환경 변수

코드 밖에서 주입되는 값. 코드에 직접 적지 않고 외부에서 받음

코드 안에 직접 — 위험

```
const apiKey = "sk-abc123def456..."
```

환경 변수 — 안전

```
const apiKey = process.env.OPENAI_API_KEY
```

[Railway 대시보드 → Variables]
OPENAI_API_KEY = sk-abc123def456...
↓ (서버 시작 시 자동 주입)
[BE 코드] process.env.OPENAI_API_KEY로 읽음

환경 변수의 핵심: 코드는 GitHub에 공개해도 값은 비공개. 코드에 키를 직접 쓰면 GitHub에 키가 노출된다.

5. 빌드와 배포

코드를 공개하는 2단계

단계	의미
빌드 (Build)	코드를 실행 가능한 형태로 변환
배포 (Deploy)	빌드된 결과물을 서버에 올림



빌드 실패와 배포 실패는 다른 문제. 빌드 실패는 **코드 자체**의 문제, 배포 실패는 **서버 설정**의 문제. 로그 위치도 다름.

6. API 엔드포인트

BE가 외부 요청을 받는 주소

```
https://my-be.up.railway.app/api/generate
↑ ↑
도메인 엔드포인트
```

엔드포인트별로 다른 일을 함:

```
POST /api/generate → 카피 생성
GET /api/history → 이전 카피 조회
POST /api/feedback → 피드백 받기
GET /health → 서버 살아있는지 확인 (헬스체크)
```

헬스체크(`/health`) 엔드포인트: 배포 직후 BE가 살아있는지 확인하는 표준 패턴. 실습 Step 1에서 확인한다.

보너스: 브라우저 개발자 도구 (F12)

브라우저에서 내부에서 일어나는 일을 보는 도구

탭	용도
Console	에러 메시지 확인 (CORS 에러도 여기)
Network	HTTP 요청·응답 추적
Application	환경 변수 노출 여부 확인

실습 Step 3에서 CORS 에러를 Console 탭에서 복사. F12 누르고 Console 클릭하는 동작이 표준.

단정 2

**AI에게 명령을 정확히 내리려면
개발 어휘가 필요하다.
코딩이 아닌 어휘다.**

어휘를 알면 명령이 정확해진다

어휘 없음 (모호)	어휘 있음 (정확)
"안 돼요"	"HTTP 요청이 CORS 에러로 차단됨"
"환경 설정 어떻게 해요"	"Railway Variables에 OPENAI_API_KEY 추가"
"빌드 안 됨"	"Railway 빌드 로그에 PORT 관련 에러"
"도메인 이상해요"	"FE는 vercel.app, BE는 railway.app — Cross-Origin"

안티패턴

"배포가 안 돼요. 무엇이 문제일까요?"
→ AI가 추측 (10개 가능성 나열)

어휘 기반 운영

"Railway 빌드 단계에서 'Cannot find module' 에러.
BE 코드의 src/lib/openai-client.ts에서 발생.
빌드 로그 풀 메시지 첨부."
→ AI가 1회 해결

챕터 용어 해설

9개 어휘 — 본 챕터의 핵심

HTTP 요청·응답

FE와 BE의 통신 패턴

도메인

서비스의 사람이 읽는 주소

포트

서버 내부의 통신 문

환경 변수

코드 밖에서 주입되는 값

빌드

코드를 실행 가능한 형태로 변환

배포

빌드 결과물을 서버에 올림

API 엔드포인트

BE가 요청을 받는 주소

헬스체크

BE 살아있는지 확인하는 표준

개발자 도구 (F12)

브라우저 내부 동작을 보는 도구

— 이론 3

Vercel과 Railway, 환경 변수

두 플랫폼 역할 + 환경 변수 3종류

- 01 두 플랫폼의 역할을 구분한다
- 02 환경 변수 3종류를 외운다
- 03 .gitignore의 의미를 안다

두 플랫폼의 역할

플랫폼	역할	무료 티어
Vercel	FE 배포	본 학습에 충분
Railway	BE 배포 (Node.js, Python, DB 모두)	월 5달러 크레딧

두 플랫폼 공통점:

- GitHub 연동 자동 배포
- 코드 push → 빌드 → URL 발급 자동

Railway 무료 티어 한계: 본 학습 후 본격 운영 시 크레딧 소진 가능. 결제 필요할 수 있다. 사전 인지가 다음 단계의 비용 통제 결정에 도움.

두 플랫폼의 분담

GitHub push 한 번 → Vercel/Railway 자동 분담 배포



환경 변수 3종류

비밀 / 공개 / 로컬 — 등록 위치도 다름

#	종류	예시 + 등록 위치	노출 여부
1	BE 비밀	<code>OPENAI_API_KEY</code> Railway Variables	비공개
2	FE 공개	<code>NEXT_PUBLIC_API_URL</code> Vercel Environment Variables	공개
3	로컬 개발	<code>.env.local</code> 본인 노트북만	비공개

3

이론 3 · 환경 변수 1종

1종류: BE 비밀 정보

`OPENAI_API_KEY` 같은 비밀

- Railway 대시보드 Variables 탭에 등록
- 절대 GitHub에 올리지 않음
- BE 코드만 `process.env`로 접근

3

이론 3 · 환경 변수 2종

2종류: FE 공개 정보

`NEXT_PUBLIC_API_URL` 같은 공개 정보

- Vercel 대시보드 Environment Variables에 등록
- NEXT_PUBLIC_ 접두사 = 브라우저로 노출되는 변수
- 비밀 정보는 절대 이 접두사로 두지 않음

공개라고 해서 적어도 된다는 뜻은 아님. 공개되어도 무관한 것만 둔다. BE URL은 공개되어도 무관 (어차피 누구나 호출 시도 가능).

3

이론 3 · 환경 변수 3종

3종류: 로컬 개발용

`.env.local` 파일

- 본인 노트북에서만 사용
- .gitignore에 명시
- git이 추적하지 않음
- 실수로 git add . 해도 안전

세 번째 단정

API 키는 환경 변수에 두고 .gitignore에 .env.local을 명시한다

원칙

코드는 GitHub에 공개해도 값은 비공개. 환경 변수를 통한 분리가 안전.

귀결

실수로 git add . 해도 .env.local이 추적되지 않으므로 안전. 키 노출 위험 없음.

.gitignore 표준 항목

Git이 추적하지 않을 파일 목록 — 비밀 노출 방지

```
.env  
.env.local  
.env.production  
*.pem  
*.key
```

.env* 모든 환경 변수 파일은 Git 추적 X. ***.pem** / ***.key** 인증서·키 파일도 동일.

안티패턴 vs 안전한 배포

안티패턴

"OpenAI API 키를 코드에 직접 넣고
GitHub에 푸시했다"
→ 봇이 즉시 발견. 무한 호출.
→ 청구서 폭발

안전한 배포

"API 키는 환경 변수에 두고
.gitignore에 .env.local 명시"
→ 코드는 GitHub 공개, 키는 비공개
→ 안전

키 노출 시 대응

만약 실수로 GitHub에 키를 푸시했다면

단계	작업
1	즉시 키 회수 (OpenAI 대시보드에서 삭제)
2	새 키 발급
3	git 히스토리에서 제거 (git filter-branch 등)
4	환경 변수로 재설정

GitHub은 공개 push 후 붓이 수초 안에 키 탐지. 키 삭제는 분 단위로 빠를수록 안전.

3

챕터 용어 해설

5개 어휘 — 본 챕터의 핵심

용어	정의
Vercel	FE 배포 플랫폼. Next.js와 통합 우수
Railway	풀스택 배포 플랫폼. BE·DB·워커
NEXT_PUBLIC_	Next.js에서 브라우저 노출 접두사
.gitignore	Git이 추적하지 않을 파일 목록
자동 재배포	환경 변수 변경 시 자동 트리거

— 이론 4

CORS — 보안 정책의 기획 관점

도메인이 다른 차단되는 이유와 화이트리스트 의사결정

- 01 CORS가 왜 존재하는지 설명
- 02 화이트리스트 정책의 의사결정 근거
- 03 첫 만남이 통과 의례인 이유

CORS의 정의

CORS (Cross-Origin Resource Sharing): 서로 다른 도메인 간 통신을 *제한*하는 브라우저 보안 정책.

도메인이 다름:

- FE: myapp.vercel.app
- BE: myapp-production.up.railway.app

↓

브라우저가 통신 차단

↓

"blocked by CORS policy" 에러

2 챕터에서 다룬 "도메인" 개념과 직접 연결. 도메인이 다르면 CORS 정책이 발동.

CORS가 왜 존재하는가

CORS는 불편한 제약이 아니라 필요한 보호 장치.

CORS가 없으면	일어나는 일
악성 사이트가 본인 BE 호출 가능	사용자가 본인 사이트 로그인 상태에서 다른 탭의 악성 페이지가 본인 BE를 마음대로 호출
본인 API 무단 사용	다른 사이트가 본인 BE를 자기 서비스에 무료로 활용
청구서 폭발	누가 호출하는지 통제 불가

브라우저가 기본으로 차단하는 이유: 사용자 보호. 본인 사이트가 신뢰할 도메인을 명시해야 통신 허용.

CORS의 두 가지 정책

본 학습의 핵심 기획 결정

정책	설정	트레이드오프
모든 도메인 허용	Access-Control-Allow-Origin: *	편리 / 누구나 호출 가능
화이트리스트	특정 도메인만 허용	본인 FE만 호출 / 신규 도메인마다 설정

본 학습은 화이트리스트 선택. 본인 Vercel URL + 로컬 개발용 localhost만 허용. 다른 사이트가 호출 시도하면 차단.

화이트리스트 정책의 효과

허용 도메인 명시 → 다른 도메인 자동 차단 → BE 자원 보호

허용 도메인 정의:

- https://myapp.vercel.app (배포된 FE)
- http://localhost:3000 (로컬 개발)

↓

다른 도메인에서 호출 시도

↓

브라우저가 자동 차단

↓

본인 BE 자원 보호

단정 4

**CORS는 보안을 위해 막히는데
이것은 정상이다.**

통과 의례인 이유

로컬에서 동작 → 배포 후 차단 → 모든 풀스택 빌드자가 만나는 패턴

- 1 로컬에서는 FE도 BE도 **localhost**
→ 같은 도메인이므로 CORS 안 막힘



- 2 배포 후 FE는 **vercel.app**, BE는 **railway.app**
→ 다른 도메인이 되면서 CORS 차단 시작



- 3 CORS 설정 필요한 시점 = 첫 배포 직후



- 4 모든 풀스택 빌드자가 한 번 만나는 패턴

로컬에서 동작하던 게 배포 후 *안 됨*. 좌절 포인트지만 정상 흐름. CORS 에러를 만나면 "*배포가 잘됐다는 신호*"로 받아들일 것.

이론 4 · 해결 두 갈래

CORS 해결의 두 갈래

BE에 "이 FE 도메인은 허용한다"를 명시

갈래	방법	본 학습 선택
FE 변경	같은 도메인에 BE 배포 (역방향 프록시 등)	복잡
BE 변경	BE에 CORS 헤더 추가	선택

자율 디버깅 적용

자료 7의 3원칙 그대로

- 1 브라우저 콘솔에서 에러 풀 메시지 복사
F12 → Console 탭
"blocked by CORS policy..." 복사



- 2 자율 디버깅 3원칙 명령



- 3 수정 후 자동 재배포



- 4 FE 새로고침 → 에러 사라지면 통과

이론 4 · 표준 명령

CORS 해결 표준 명령

BE 코드에 CORS 화이트리스트 추가 + 다른 도메인 차단 검증

이 BE 코드에 CORS 설정을 추가해줘.
허용할 FE 도메인은 `https://{Vercel URL}`.
로컬 개발용 `http://localhost:3000`도 허용해줘.

설정 후 다른 도메인에서 호출 시 차단되는지도
함께 점검해줘.

"다른 도메인 차단 확인"이 핵심. 화이트리스트가 *제대로 좁게* 설정됐는지 검증.

검증 체크리스트 + 기획적 운영

체크리스트 4개 + 안티 vs 기획 비교

CORS 설정의 검증 체크리스트

항목	확인
본인 FE 도메인 허용	필수
로컬 개발용 허용	필수
와일드카드(*) 사용 안 함	필수
알 수 없는 도메인 차단	필수

안티패턴 vs 기획적 운영

안티패턴

Access-Control-Allow-Origin: * 설정 → 모든 도메인 허용 → BE 무단 사용 가능

기획적 운영

화이트리스트로 본인 FE만 허용. 신규 도메인 추가 시 명시적으로 설정.

4 **챕터 용어 해설**

4개 어휘 — 본 챕터의 핵심

용어	정의
CORS	다른 도메인 간 통신을 제한하는 브라우저 정책
화이트리스트	허용할 도메인 명시. 그 외 차단
Cross-Origin	서로 다른 도메인. 기본 차단 대상
통과 의례	첫 배포 후 누구나 만나는 CORS 에러

분석 도구 3종 세 가지 다른 질문에 답하기

Clarity / Sentry / 이벤트 추적 — 세 도구가 함께 만드는 의사결정 흐름

- 01 각 도구가 답하는 다른 질문 구분
- 02 사용자 들어오기 전 셋업의 시점성
- 03 세 도구가 왜 한 도구로 안 되는지
- 04 세 도구가 함께 만드는 의사결정 흐름

이론 5 · 필요성

분석 도구가 왜 필요한가

검증된 MVP 사용자 노출 단계에서 작업자가 모를 일이 일어남

사용자 행동: - 어디서 막혔는가? (모름) - 어느 버튼이 안 보였는가? (모름) - 어떤 에러를 만났는가? (사용자가 보고 안 함) - PRD 가설이 진짜 통과하는가? (체감만)

사용자는 피드백을 안 줌. 안 좋은 경험이면 그냥 떠나고 다시 안 옴. 작업자가 데이터로 자동 수집하지 않으면 그 사용자의 경험은 영원히 모름.

단정 5

**사용자가 들어오기 전에 분석 도구를 셋업한다.
들어온 후 셋업하면 그 사용자 데이터는 회수 불가능
하다.**

이론 5 · 1명 사용자

1명 사용자 = 회수 불가능한 자산

셋업 안 한 상태 vs 셋업 한 상태

셋업 안 한 상태

사용자 1명 진입 → 사용 → 떠남 ↓ 그 1명이 어디서 막혔는지 영원히 모름 ↓ 다음 사용자가 같은 곳에서 막혀도 같은 문제 반복

셋업 한 상태

사용자 1명 진입 → 모든 행동 자동 기록 ↓ 나중에 세션 영상 재생 가능 ↓ 막힌 지점 발견 → 수정 → 다음 사용자에게 적용

분석 도구 3종의 다른 질문

세 도구는 서로 다른 질문에 답함. 한 도구로 다 해결 안 됨.

도구	답하는 질문	기획 가치
Microsoft Clarity	사용자가 어디서 막히는가?	UX 가설 검증
Sentry	우리가 모르는 에러가 얼마나 있는가?	품질 신호
이벤트 추적	PRD 가설이 실제로 통과하는가?	가설 검증

세 질문이 각각 다른 의사결정 영역. UX 수정 / 코드 버그 수정 / PRD 갱신. 한 도구로 모든 결정을 내릴 수 없는 이유.

Microsoft Clarity — 행동 분석

답하는 질문: "사용자가 어디서 막히는가?"

기능	의사결정 활용
세션 리플레이 (화면 영상)	막힌 지점 정확한 위치 발견
히트맵 (클릭/스크롤 분포)	어느 버튼이 안 보이는지 확인
페이지 체류 시간	어디서 머뭇거리는지

시드 6번 예시: 셀러가 "키워드 5개 입력" 단계에서 3개만 입력하고 머뭇거리는 영상이 보이면 → 5개 강제 규칙이 너무 뻑뻑한 신호. PRD의 가설 수정 근거.

Clarity의 핵심 비교

셋업 안 한 경우 vs Clarity 있는 경우

셋업 안 한 경우	CLARITY 있는 경우
사용자가 "안 좋았어요"	사용자가 어디서 막혔는지 정확한 화면 영상
추측으로 수정	데이터 기반 수정
같은 문제 반복	한 번 수정 후 효과 측정 가능

Sentry — 에러 자동 수집

답하는 질문: "우리가 모르는 *에러*가 얼마나 있는가?"

사용자는 에러를 **알려주지 않음**. 본인이 만든 코드의 빈틈을 사용자가 발견하지만 보고 의무가 없음.



시드 6번 예시: OpenAI API 호출이 5% 사용자에게 timeout 발생 → Sentry가 알려주면 retry 로직 추가 결정. 알리지 않았다면 5% 사용자는 "안 됨"만 경험하고 떠남.

Sentry가 알려주는 것

항목	의사결정 활용
에러 발생 횟수와 영향 사용자 수	우선순위 결정
스택 트레이스 (정확한 코드 위치)	자율 디버깅 즉시 적용
발생 컨텍스트 (페이지, 입력값)	재현 가능성 확보

시드 6번 예시: OpenAI API 호출이 5% 사용자에게 timeout 발생 → Sentry가 알려주면 retry 로직 추가 결정. 알리지 않았다면 5% 사용자는 "안 됨"만 경험하고 떠남.

Sentry의 본질 — 안전망

셋업 안 한 경우 vs Sentry 있는 경우

셋업 안 한 경우	SENTRY 있는 경우
에러 발생 → 사용자 떠남 → 작업자 모름	에러 발생 즉시 알림 → 패턴 분석 → 수정
사용자 신뢰 침식	빠른 대응으로 신뢰 회복

이벤트 추적 — PRD 가설 검증

답하는 질문: *"PRD 가설이 실제로 통과하는가?"*

자료 2-3에서 만든 PRD 핵심 가설을 실제 사용자 행동 데이터로 검증.

시트 6번 PRD 가설 예시:

"카피 작성 시간 1-2시간 → 5분 이내"
"카피 후보 사용률 70% 이상"

이벤트 추적 없으면:

사용자 진입 → 사용 → 떠남
↓
가설이 통과했는가? 모름
↓
PRD 갱신 근거 없음

이벤트 추적 표준 3개

본 학습에서 추적할 핵심 이벤트

이벤트	시드 6번 적용	답하는 질문
페이지 진입	카피 입력 화면 방문	누가 들어오는가
핵심 버튼 클릭	"카피 생성" 클릭	핵심 기능 시도하는가
핵심 기능 완료	카피 1개 복사	가설 통과하는가

시드 6번 가설 검증: 100명 진입 → 70명이 카피 생성 클릭 → 50명이 복사까지 진행. $50/100 = 50\%$ 완료율. PRD 목표 70% 미달 → PRD 갱신 또는 UX 수정 결정.

이벤트 추적 vs Clarity

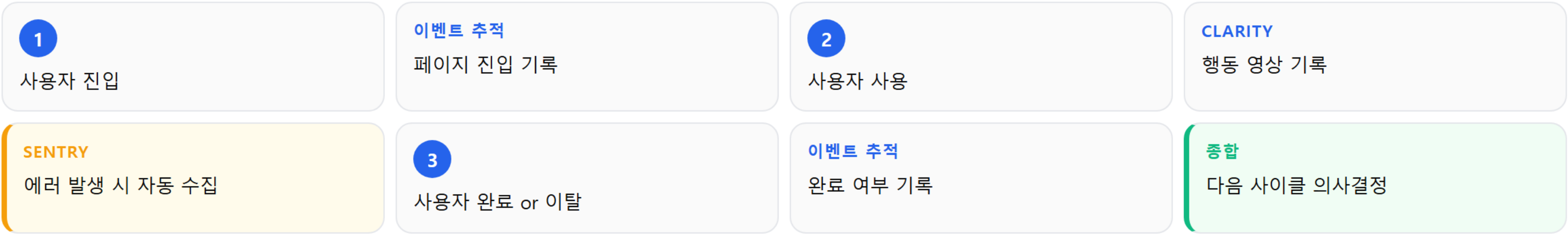
두 도구는 상호 보완 — 숫자 + 영상

항목	이벤트 추적	CLARITY
데이터 형태	숫자 (몇 명이 했는가)	영상 (어떻게 했는가)
답하는 차원	가설 통과율	막힌 지점
의사결정	PRD 갱신	UX 개선

두 도구가 상호 보완. 이벤트 추적이 가설 미달을 알려주면 → Clarity로 왜 미달했는지 영상 확인.

세 도구가 함께 만드는 의사결정 흐름

이벤트 추적 + Clarity + Sentry → 다음 사이클 의사결정



Sentry 에러 패턴 → 코드 버그 수정 · Clarity 막힘 지점 → UX 개선 · 이벤트 추적 가설 미달 → PRD 갱신

이론 5 · 종합 비교

세 도구 종합 비교표

비용 + 셋업 + 질문 + 의사결정

도구	비용	셋업 비용	답하는 질문	의사결정
Clarity	무료	코드 한 줄	어디서 막히는가	UX 개선
Sentry	무료 티어 충분	SDK 설치	무슨 에러 있는가	버그 수정
이벤트 추적	Clarity 안에서	코드 3-5줄	가설 통과하는가	PRD 갱신

셋업 시점이 중요한 이유

다음 단계 직전 vs 발송 후

[다음 단계(노출) 직전 셋업]

사용자 진입 → 모든 행동 기록
↓
다음 사이클의 의사결정 근거 확보

[다음 단계 발송 후 셋업]

사용자 진입 → 행동 기록 X
↓
그 사용자 데이터 영원히 회수 불가
↓
"왜 안 썼는가" 추측만 가능

1명의 사용자가 1개 세션을 만들면 그 세션은 한 번뿐. 셋업 안 된 상태로 흘려보내면 평생 못 본다.

안티패턴 vs 올바른 순서

분석 도구 셋업의 올바른 시점

안티패턴

"다음 단계 발송 후에 Clarity를 깬다"
"먼저 노출하고 반응 보면서 도구 추가"
→ 첫 사용자들 데이터 영원히 손실

올바른 순서

"본 단계에서 Clarity, Sentry, 이벤트 추적
모두 셋업 → 다음 단계에서 발송"
→ 첫 사용자부터 모든 데이터 확보

5 **챕터 용어 해설**

6개 어휘 — 본 챕터의 핵심

Microsoft Clarity

무료 행동 분석 도구. 세션 리플레이

Sentry

에러 자동 수집과 알림 도구

이벤트 추적

사용자의 특정 행동을 데이터로 기록

세션 리플레이

사용자 화면을 영상처럼 재생

히트맵

클릭/스크롤 분포 시각화

가설 검증

PRD 가설을 실제 행동 데이터로 측정